COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/26	**DATE SUBMITTED:**	09/10/26
NAME:	Cruz, Elijah Villaruel, Prince Benjamin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

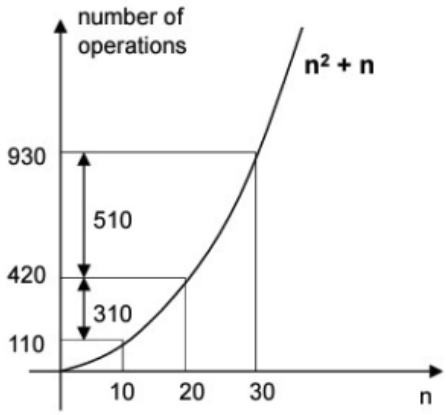
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **O(n^2)**

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worst case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

VILLARUEL

- In the worst case, the total number of comparisons is m x n, which is equivalent to n^2 when both arrays are the same size. In the graph, it shows quadratic growth based on the equation n^2 + n, where the operations clearly increase from 110 at n = 10 to 930 at n = 30.

CRUZ

- The graph shows the worst-case number of operations for an algorithm as the array size n grows, following the function $n^2 + n$. As n increases from 10 to 20 to 30, the operations rise from 110 to 420 to 930, and the curve gets steeper because each step adds more than the last (+310, then +510). The n^2 term dominates as n grows, so the worst-case time complexity is $O(n^2)$, meaning the algorithm slows down quickly on large inputs.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	**1 microseconds**	`Time taken to search = 1 microseconds` `Student (202 409) needs vaccination.`	The program searched the list of 1,000 students. And it took just 1 microsecond to finish the process. Although, the student (997 936) was most likely not in the list, because the program said they "need vaccination," and that is the answer it gives when it can't find the student.
10000	**2 microseconds**	`Time taken to search = 1 microseconds` `Student (997 936) needs vaccination.`	The list is now 10 times bigger, but the time

			stayed at 1 microsecond. This can vary but does happen when the list gets 10 times larger but only requires 3 more or so steps. The three extra steps took only a few nanoseconds, so it still shows as a 1 millisecond output. The same student appeared again and was probably not found.
100000	**3 microseconds**	`Time taken to search = 3 microseconds` `Student (202 409) needs vaccination.`	The list is now 100 times bigger than the first test, but the time only went up to 3 microseconds. The small jump from 1 to 3 milliseconds comes from the extra steps, plus a bit of a probable normal timing noise. A bigger list may also not fit as well in the computer's fast memory, which makes each step slightly slower than the first ones

OUTPUT

```
Output

Time taken to search = 1 microseconds
Student (997 936) needs vaccination.
Time taken to search = 1 microseconds
Student (997 936) needs vaccination.
Time taken to search = 3 microseconds
Student (997 936) needs vaccination.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis
For each of the problems below, provide the following:
Theoretical Analysis
Experimental Analysis
Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Theoretical Analysis
Let n = the number of elements in the array. We can count the comparison A[i] == A[j].
Worst case: all elements are seen to be unique, so the function never returns early.

i	Comparisons in the inner loop
0	n - 1
1	n - 2
2	n - 3
…	…
n - 2	1

Total = (n−1) + (n−2) + … + 1 = n(n−1)/2
The worst case is O(n²).
Best case: A[0] equals A[1], so it returns after 1 comparison. This is O(1).

n	Comparisons (worst case)
10	45
20	190
30	435

COMPUTER ENGINEERING

QUEZON CITY

Experimental Analysis

CODE

CMakeLists.txt | main.cpp

```cpp
#include <iostream>
#include <chrono>

// returns true if all elements are unique
bool isUnique(int A[], int n, long long &count) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            count++;
            if (A[i] == A[j]) return false;
        }
    }
    return true;
}

int main() {
    int sizes[] = {1000, 2000, 4000, 8000};
    for (int n : sizes) {
        int *A = new int[n];
        for (int i = 0; i < n; i++) A[i] = i;   // all unique = worst case
        long long count = 0;
        auto start :time_point<steady_clock> = std::chrono::steady_clock::now();
        isUnique(A, n, [&] count);
        auto end :time_point<steady_clock> = std::chrono::steady_clock::now();
        auto time :long long = std::chrono::duration_cast<std::chrono::microseconds>( d: end - start).count();
        std::cout << "n = " << n << " | comparisons = " << count << " | time = " << time << " microseconds" << std::endl;
        delete[] A;
    }
    return 0;
}

```

OUTPUT

```
C:\Users\Elijah\CLionProjects\SP\cmake-build-debug\SP.exe
n = 1000 | comparisons = 499500 | time = 965 microseconds
n = 2000 | comparisons = 1999000 | time = 2610 microseconds
n = 4000 | comparisons = 7998000 | time = 10472 microseconds
n = 8000 | comparisons = 31996000 | time = 41867 microseconds

Process finished with exit code 0
```

```
PS C:\Users\TIPQC\Desktop\Villaruel\PAIR HOA> cd "c:\Users\TIPQC\
n = 1000 | comparisons = 499500 | time = 919 microseconds
n = 2000 | comparisons = 1999000 | time = 3606 microseconds
n = 4000 | comparisons = 7998000 | time = 14667 microseconds
n = 8000 | comparisons = 31996000 | time = 59089 microseconds
```

CRUZ

n	Comparisons	Time
1000	499500	607
2000	1999000	2556
4000	7998000	10243
8000	31996000	41495

VILLARUEL

n	Comparisons	Time
1000	499500	919
2000	1999000	3606
4000	7998000	14667
8000	31996000	59089

ANALYSIS AND COMPARISON

CRUZ

The formula n(n−1)/2 for n = 1,000 gives us 499500, which is exactly what the program counted. The counts for the other sizes match the formula too, so our theoretical analysis is correct. When n doubles, the number of comparisons becomes about 4 times bigger as we can observe. The running time also grows by about 4 times each time n doubles. The time is not perfectly 4× because the computer's speed changes a little from run to run. Both the theory and the experiment show that the code for our Unique(A) is **$O(n^2)$**, so it gets slow when the array is large.

VILLARUEL

In our theoretical analysis, I determined that the isUnique algorithm has a worst case time complexity of O(n^2) because comparing every unique pair requires n(n−1)/2 comparison operations. Experimentally, I noticed that as n doubled from 1000 to 8000, the total comparison count grew quadrupled from 499,500 to 31,996,000. Consequently, our measured runtime scaled quadratically, increasing from hundreds of microseconds at n = 1000 to tens of thousands of microseconds at n = 8000 on both machines.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Theoretical Analysis

rpower(x, n)

- Each call lowers n by 1: $n \rightarrow n-1 \rightarrow n-2 \rightarrow \ldots \rightarrow 0$.
- For every call, n decreases by 1, so there are n recursive calls
- Time complexity: O(n)

brpower(x, n)

- Each call cuts n in half: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \ldots \rightarrow 0$.
- So there are only about $\log_2 n + 1$ calls.
- Time complexity: O(log n)

n	Rpower calls	Brpower calls
8	9	5
16	17	6
1024	1025	12

Experimental Analysis

CODE

main2.cpp > brpower(double, int)

```cpp
#include <iostream>
#include <chrono> // for timing

// linear power - subtracts 1 each time
double rpower(double x, int n) {
    if (n == 0) return 1.0;
    return x * rpower(x, n - 1);
}

// binary power - divides n by 2 each time
double brpower(double x, int n) {
    if (n == 0) return 1.0;
    if (n % 2 != 0) { // odd
        double y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else { // even
        double y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    double x = 2.0;
    int test_values[] = {8, 16, 1024};

    for (int n : test_values) {
        std::cout << "n = " << n << " \n";

        // Measure rpower runtime
        auto start1 = std::chrono::high_resolution_clock::now();
        double res1 = rpower(x, n);
        auto end1 = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::micro> time1 = end1 - start1;

        // Measure brpower runtime
        auto start2 = std::chrono::high_resolution_clock::now();
        double res2 = brpower(x, n);
        auto end2 = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::micro> time2 = end2 - start2;

        std::cout << "rpower time : " << time1.count() << " microseconds\n";
        std::cout << "brpower time: " << time2.count() << " microseconds\n\n";
    }

    return 0;
}
```

CODE OUTPUT

```
PS C:\Users\TIPQC\Desktop\Villaruel\PAIR HOA> c
n = 8
rpower time : 0.1 microseconds
brpower time: 0.1 microseconds

n = 16
rpower time : 0.1 microseconds
brpower time: 0.1 microseconds

n = 1024
rpower time : 28.1 microseconds
brpower time: 0.2 microseconds
```

```
PS C:\Users\TIPQC\Documents\cruz> cd "c:\Use
PS C:\Users\TIPQC\Documents\cruz> cd "c:\Use
n = 8
rpower time : 0.2 microseconds
brpower time: 0.1 microseconds

n = 16
rpower time : 0.1 microseconds
brpower time: 0.1 microseconds

n = 1024
rpower time : 27.9 microseconds
brpower time: 0.1 microseconds

PS C:\Users\TIPQC\Documents\cruz>
```

VILLARUEL

n	Time (rpower/brpower)
8	0.2 / 0.1
16	0.1 / 0.1
1024	28.1 / 0.2

CRUZ

n	Time (rpower/brpower)
8	0.2 / 0.1
16	0.1 / 0.1
1024	27.9 / 0/2

ANALYSIS AND COMPARISON

VILLARUEL

I evaluated the runtime performance of rpower and brpower by comparing our theoretical models with our experimental pair results. Theoretically, rpower runs in O(n) time due to its n + 1 linear calls, while brpower runs in O(log n) time by cuttin n in half each step. Experimentally, I observed that while both functions took 0.1-0.2 microseconds for small inputs (n = 8, 16) increasing n to 1024 caused rpower to jump to 28 microseconds while brpower stayed under 0.2 microseconds .

CRUZ

The results show that brpower is much faster than `rpower` when n gets large. For small values like 8 and 16, both take about the same time (around 0.1 to 0.2 microseconds), so we can't really tell them apart. But at n = 1024, rpower takes about 27.9 microseconds while `brpower` still takes only 0.1. This happens because rpower reduces n by 1 each time, so it makes about 1024 calls during the iteration, while brpower cuts n in half each time, so it only needs about 11 calls. In short, brpower (O(log n)) scales far better than rpower (O(n)), and the bigger n is, the bigger the difference.

CONCLUSION

CRUZ

In conclusion, this laboratory tackles algorithm analysis and the importance of an algorithm's structure, which proves to be relevant to how quickly any program can run under different time complexities. As observed, the programs given and completed for both the supplementary and procedural tasks ran at different speeds when compared. Upon further analysis, we can infer and imply that the speed of the user's hardware is an additional factor in how fast a program runs, aside from its algorithm structure. In addition, the completion times were longer as the value of n increased and shorter for smaller values of n. Overall, the laboratory paves the way for learning how to properly build programs, while also giving us the opportunity to use critical thinking skills through the theoretical and experimental analysis included in the laboratory, aside from the comparison.

VILLARUEL

In completing this hands-on activity, I found the concept easy to understand but quite hard when you go deeper in terms of details and implementation. However, the supplementary activity actually helped me gain a little bit of

understanding especially during the comparison of both the theoretical and experimental results. I noticed that theoretically faster algorithms consistently maintain low, stable runtimes even as input sizes grow, whereas less efficient algorithms suffer exponential or linear spikes in execution time and operations. Ultimately, I think that it is a must having a theoretical and exponential analysis and comparison as it serves as a guide for users to use and compare results in order to create a working code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **GEMINI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *grammar correction, code assistance, explanation of concepts*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: *grammar correction, code assistance, explanation of concepts*

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.